

Week 8: Efficiency and High Performance Computing

PM 566: Introduction to Health Data Science

George G. Vega Yon and Kelly Street

Part I: Efficiency

Why does it matter?

Often, we will want to do the same task (or similar tasks) many times with only slight differences.

Some tasks are very “computationally expensive” meaning they require a lot of time or other resources to run.

In these cases, computational efficiency can be very important. It can mean the difference between a job that takes two hours to run as opposed two weeks!

Efficiency: Loops

We've seen various examples of loops so far in this class.

Classic examples that are common to most programming languages include `for` and `while` loops.

Other types of loops are more specific to R, such as the various `*apply` functions (`apply`, `sapply`, `lapply`, etc.).

Knowing when to use a loop and *when not to* is a critical concern in computational efficiency.

Example: Square Roots

Let's say we want to take the square roots of the numbers 1 to 10,000. We could achieve this in a number of different ways.

Here, we use a **for** loop:

```
1 forLoopRoots <- function(n){  
2   roots <- rep(NA, n)  
3   for(i in 1:n){  
4     roots[i] <- sqrt(i)  
5   }  
6   return(roots)  
7 }  
8  
9 head(forLoopRoots(10000))
```

```
[1] 1.000000 1.414214 1.732051 2.000000 2.236068 2.449490
```

Example: Square Roots (cont. 1)

Here, we'll use `sapply`:

```
1 SapplyRoots <- function(n){  
2   roots <- sapply(1:n, sqrt)  
3   return(roots)  
4 }  
5  
6 head(SapplyRoots(10000))
```

```
[1] 1.000000 1.414214 1.732051 2.000000 2.236068 2.449490
```

Example: Square Roots (cont. 2)

How fast are these methods? Let's use the `microbenchmark` package to find out!

```
1 library(microbenchmark)
2 microbenchmark(forLoopRoots(10000), SapplyRoots(10000))
```

Unit: microseconds

	expr	min	lq	mean	median	uq	max
forLoopRoots(10000)		305.081	311.641	316.3474	313.6295	318.119	451.410
SapplyRoots(10000)		1538.402	1574.482	1732.0266	1620.1970	1669.869	3202.469
neval							
100							
100							

Example: Square Roots (cont. 3)

But do we really need a loop for this job? Here, we'll use **vectorization** to perform all the calculations “at once.”

```
1 VectorizedRoots <- function(n){  
2   sqrt(1:n)  
3 }  
4  
5 head(VectorizedRoots(10000))
```

```
[1] 1.000000 1.414214 1.732051 2.000000 2.236068 2.449490
```

Example: Square Roots (cont. 4)

That's a lot faster!

```
1 library(microbenchmark)
2 microbenchmark(forLoopRoots(10000), VectorizedRoots(10000))
```

Unit: microseconds

	expr	min	lq	mean	median	uq	max
	forLoopRoots(10000)	326.360	342.4935	347.52830	345.9785	348.3565	569.080
	VectorizedRoots(10000)	11.726	20.4590	28.82382	21.2585	22.4885	760.509
neval							
100							
100							

Anytime that you can use vectorized computation, you should!
It will (almost) always save you a lot of computation time!

Use pre-made functions

Let's say there's a relatively simple operation you want to perform. You know how to write the code for it, but you're pretty sure there's also a built-in function that performs the same task. Which should you use?

Generally speaking, you should aim to write as little code as possible. Built-in functions (especially in the **base** package) are highly optimized and likely to run much faster than manual implementations.

Example: Repeated elements

Let's say we have a long vector and we want to find all of the repeated elements.

```
1 set.seed(1)
2 vec <- sample(letters, 100, replace = TRUE)
3 vec
```

```
[1] "y" "d" "g" "a" "b" "w" "k" "n" "r" "s" "a" "u" "u" "j" "v" "n" "j" "g"
[19] "i" "o" "u" "e" "i" "y" "n" "e" "e" "b" "j" "y" "l" "o" "a" "t" "c" "f"
[37] "j" "j" "f" "o" "t" "t" "z" "l" "y" "w" "f" "y" "h" "l" "y" "w" "x" "f"
[55] "z" "g" "s" "j" "f" "x" "n" "b" "m" "r" "v" "n" "f" "a" "s" "s" "h" "f"
[73] "w" "l" "f" "h" "g" "k" "q" "d" "m" "h" "y" "p" "y" "w" "n" "t" "g" "m"
[91] "v" "l" "p" "a" "m" "u" "f" "q" "i" "g"
```

Example: Repeated elements (cont. 1)

We can achieve this with two nested loops:

```
1 method1 <- function(x){  
2   # first element cannot be a duplicate  
3   out <- sapply(2:length(x), function(i){  
4     matches <- sapply(1:(i-1), function(i2){  
5       x[i2] == x[i]  
6     })  
7     return(any(matches))  
8   })  
9   out <- c(FALSE, out)  
10  return(out)  
11 }  
12  
13 vec[1:11]
```

```
[1] "y" "d" "g" "a" "b" "w" "k" "n" "r" "s" "a"
```

```
1 method1(vec)[1:11]
```

```
[1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE
```

Example: Repeated elements (cont. 2)

Or slightly faster with a loop and vectorized comparison:

```
1 method2 <- function(x){
2   # first element cannot be a duplicate
3   out <- sapply(2:length(x), function(i){
4     return(any(x[1:(i-1)] == x[i]))
5   })
6   out <- c(FALSE, out)
7   return(out)
8 }
9
10 method2(vec)[1:11]
```

```
[1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE
```

```
1 microbenchmark(method1(vec), method2(vec))
```

Unit: microseconds

	expr	min	lq	mean	median	uq	max	neval
method1(vec)		3179.181	3260.914	3518.4630	3297.548	3364.235	5949.346	100
method2(vec)		113.406	119.392	162.6655	124.886	129.642	2429.045	100

Example: Repeated elements (cont. 3)

But the real speed-up comes when we use the base function `duplicated`:

```
1 method3 <- function(x){  
2   duplicated(x)  
3 }  
4  
5 method3(vec)[1:11]
```

```
[1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE
```

```
1 microbenchmark(method2(vec), method3(vec))
```

Unit: microseconds

	expr	min	lq	mean	median	uq	max	neval
method2(vec)		115.374	119.9455	123.11480	121.2575	124.599	159.859	100
method3(vec)		2.050	2.2960	7.52104	2.4190	2.583	511.557	100

Part II: High Performance Computing

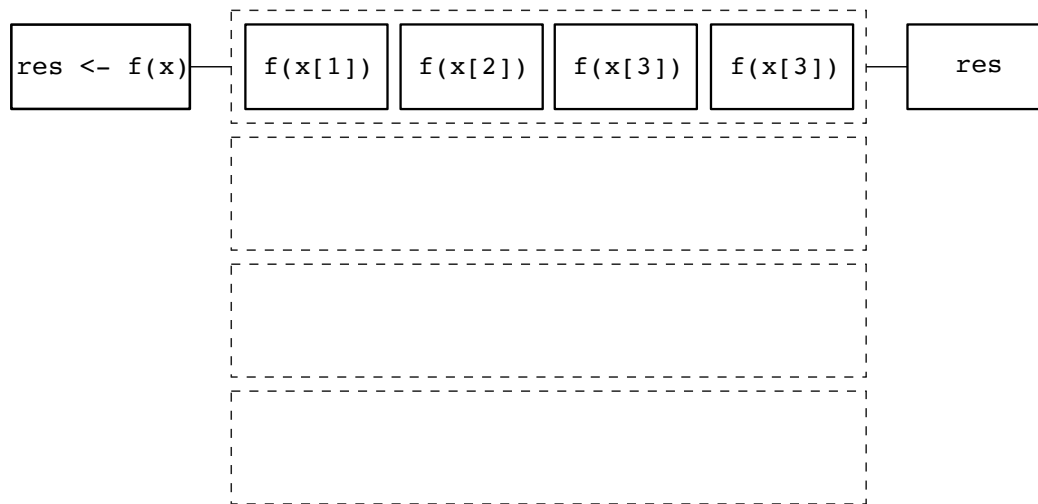
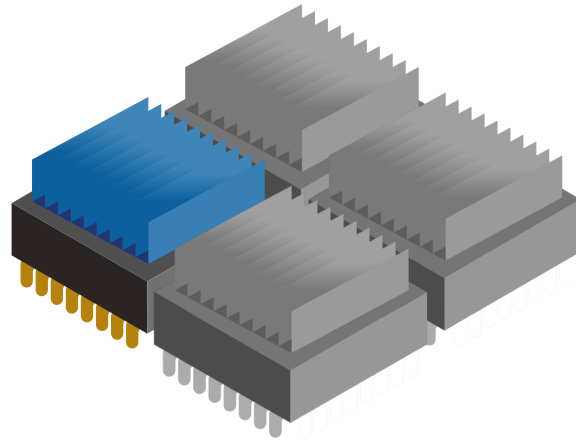
What is HPC?

High Performance Computing (HPC) can relate to any of the following:

- **Parallel computing**, i.e. using multiple resources (could be threads, cores, nodes, etc.) simultaneously to complete a task.
- **Big data** working with large datasets (in/out-of-memory).

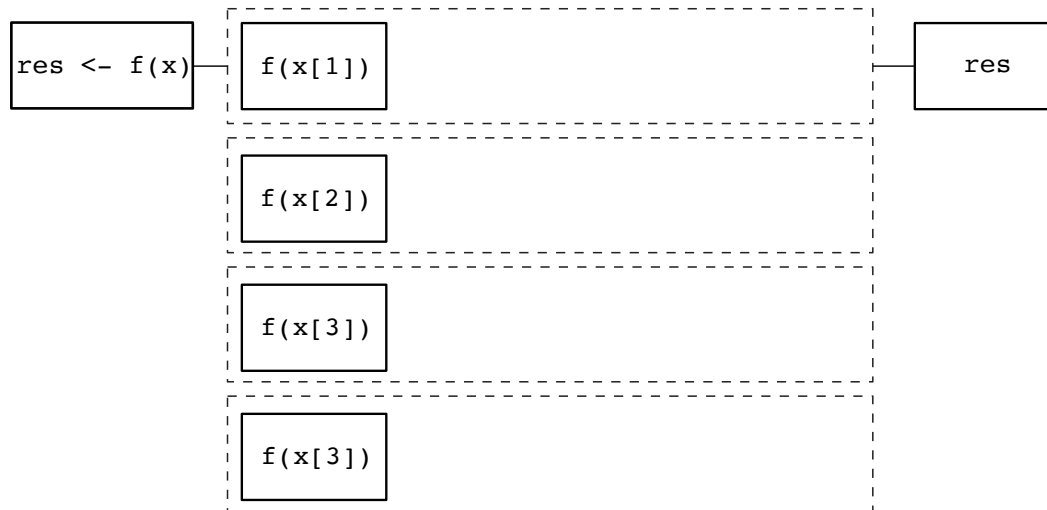
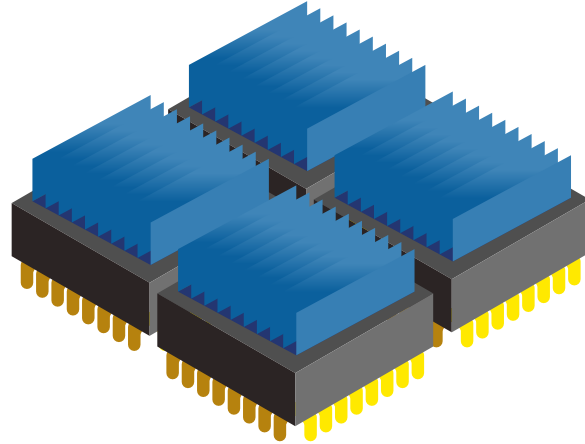
We will mostly focus on parallel computing.

Serial computation



Here we are using a single core. The function is applied one element at a time, leaving the other 3 cores without usage.

Parallel computation



In this more intelligent way of computation, we are taking full advantage of our computer by using all 4 cores at the same time. This will reduce computation time which, in the case of complicated/long calculations, can be an important speed gain.

Parallel computing: Hardware

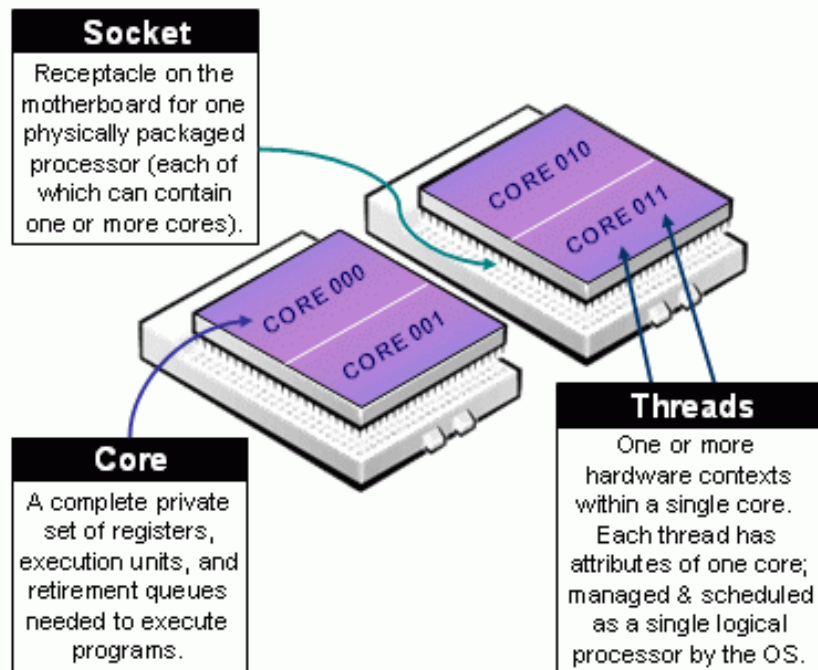
When it comes to parallel computing, there are several ways (levels) in which we can speed up our analysis. From the bottom up:

- **Thread level SIMD instructions**: Most modern processors support some level of what is called vectorization, this is, applying a single (same) instruction to streams of data. For example: adding vector **A** and **B**.
- **Hyper-Threading Technology** (HTT): Intel's hyper-threading generates a virtual partition of a single core (processor) which, while not equivalent to having multiple physical threads, does speed things up.
- **Multi-core processor**: Most modern CPUs (Central Processing Unit) have two or more physical cores. A typical laptop computer has about 8 cores.
- **General-Purpose Computing on Graphics Processing Unit** (GP-GPU): While modern CPUs have multiple cores, GPUs can hold thousands of cores. Designed for image processing, there's an increasing use of GPUs as an alternative to CPUs for scientific computing.

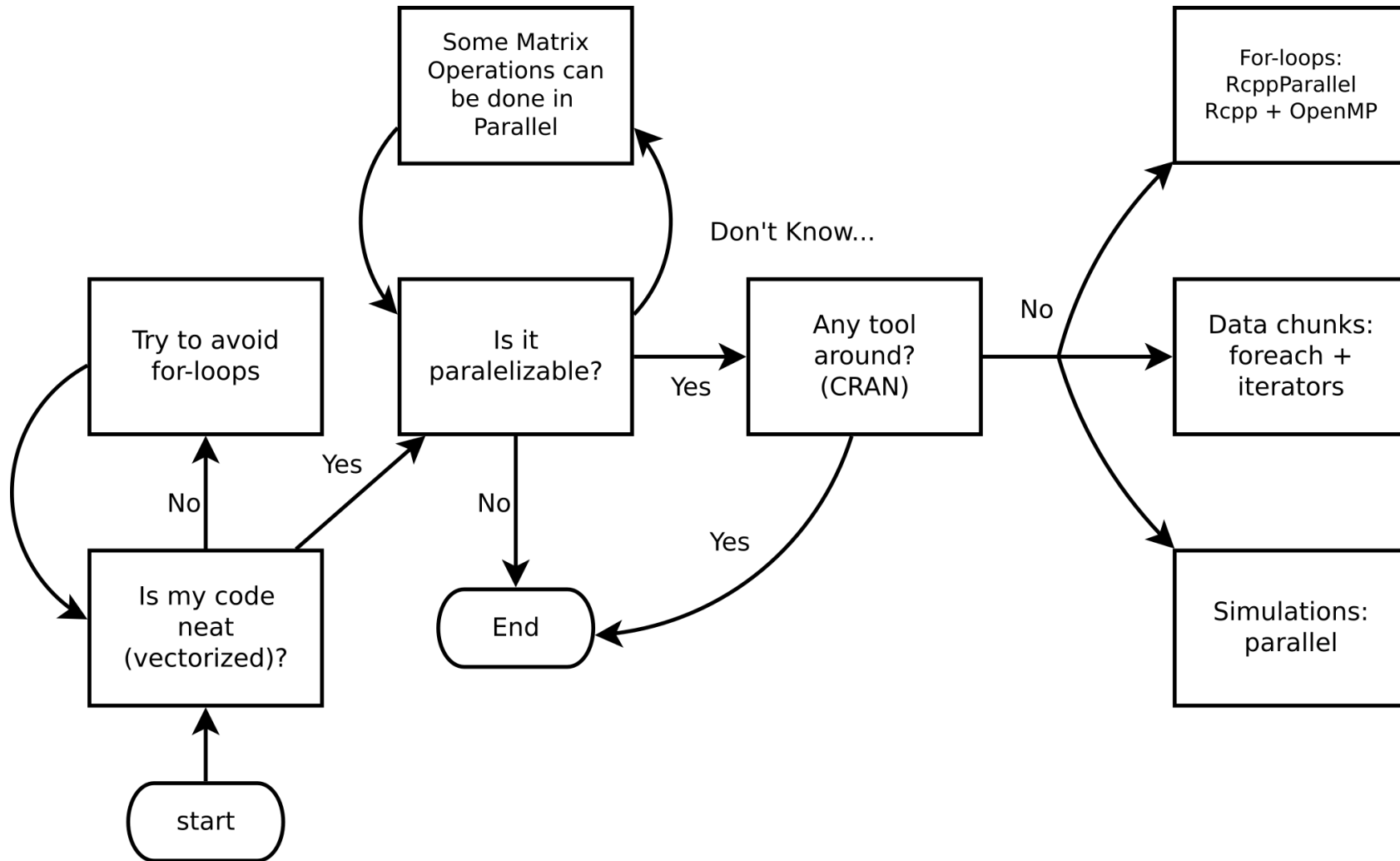
Parallel computing: Hardware (cont.)

- **High-Performance Computing Cluster** (HPC): A collection of computing nodes that are interconnected using a fast Ethernet network.
- **Grid Computing**: A collection of loosely interconnected machines that may or may not be in the same physical place, for example: HTCondor clusters.

Parallel computing: CPU components



Taxonomy of CPUs (Downloaded from https://slurm.schedmd.com/mc_support.html)



Ask yourself these questions before jumping into HPC!

Parallel computing in R

While there are several alternatives (just take a look at the [High-Performance Computing Task View](#)), we'll focus on the following R-packages for **explicit parallelism**

Some examples:

- **parallel**: R package that provides '[s]upport for parallel computation, including random-number generation'.
- **foreach**: R package for 'general iteration over elements' in parallel fashion.
- **future**: '[A] lightweight and unified Future API for sequential and parallel processing of R expression via futures.' (won't cover here)

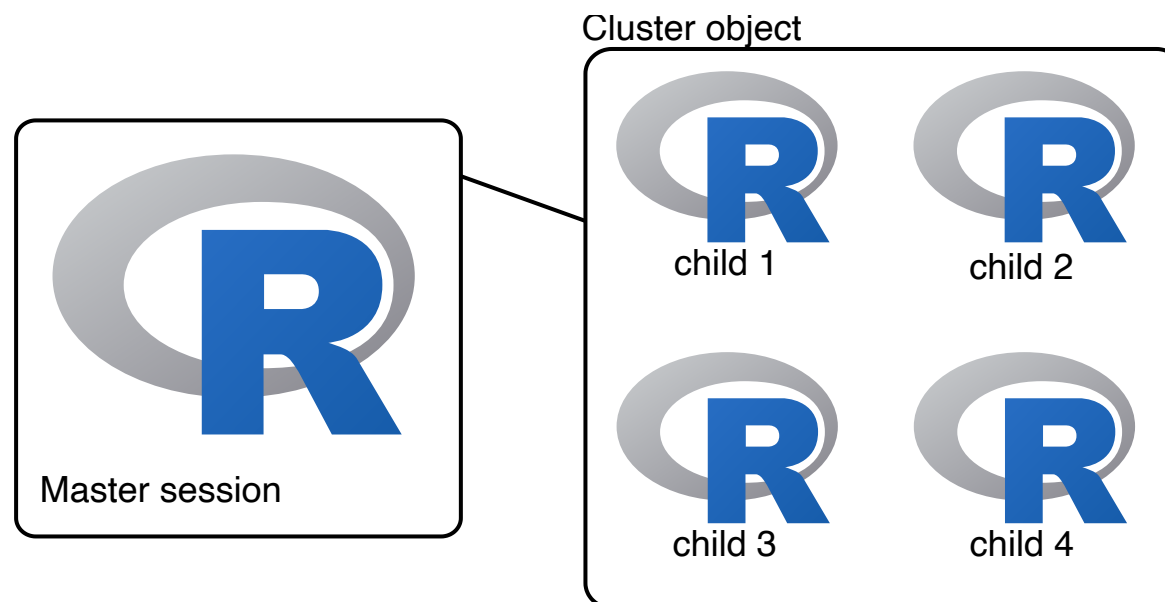
Implicit parallelism, on the other hand, are out-of-the-box tools that allow the programmer not to worry about parallelization, e.g. such as **gpuR** for Matrix manipulation using GPU, **tensorflow**

And there's also a more advanced set of options

- **Rcpp** + **OpenMP**: **Rcpp** is an R package for integrating R with C++, and OpenMP is a library for high-level parallelism for C/C++ and Fortran.
- A ton of other type of resources, notably the tools for working with batch schedulers such as Slurm, HTCondor, etc.

The **parallel** package

- Based on the **snow** and **multicore** R Packages.
- Explicit parallelism.
- Simple yet powerful idea: Parallel computing as multiple R sessions.
- Clusters can be made of both local and remote sessions
- Multiple types of cluster: **PSOCK**, **Fork**, **MPI**, etc.



Parallel workflow

(Usually) We do the following:

1. Create a **PSOCK/FORK** (or other) cluster using **makePSOCKCluster**/**makeForkCluster** (or simply **makeCluster**)
2. Copy/prepare each R session (if you are using a **PSOCK** cluster):
 - a. Copy objects with **clusterExport**
 - b. Set a seed with **clusterSetRNGStream**
3. Do your call:
 - a. Pass expressions with **clusterEvalQ**
 - b. Run a parallelized loop with **parApply**, **parLapply**, etc.
4. Stop the cluster with **clusterStop**

Ex 1: Hello world!

```
1 # 1. CREATING A CLUSTER
2 library(parallel)
3 cl <- makeCluster(4)
4 x  <- 20
5
6 # 2. PREPARING THE CLUSTER
7 clusterSetRNGStream(cl, 123) # Equivalent to `set.seed(123)` (not necessary for this example)
8 clusterExport(cl, "x")
9
10 # 3. DO YOUR CALL
11 clusterEvalQ(cl, {
12   paste0("Hello from process #", Sys.getpid(), ". I see x and it is equal to ", x)
13 })
```

```
[[1]]
[1] "Hello from process #85411. I see x and it is equal to 20"
```

```
[[2]]
[1] "Hello from process #85410. I see x and it is equal to 20"
```

```
[[3]]
[1] "Hello from process #85412. I see x and it is equal to 20"
```

```
[[4]]
[1] "Hello from process #85409. I see x and it is equal to 20"
```

```
1 # 4. STOP THE CLUSTER
2 stopCluster(cl)
```

Ex 1: Hello world! (redux)

Multi-core versions of the `*apply` functions are simpler, but may not work on Windows machines.

```
1 # 1. SETUP
2 library(parallel)
3 x <- 20
4
5 # 2. DO YOUR CALL
6 mclapply(1:4, function(i){
7     paste0("Hello from process #", Sys.getpid(), ". I see x and it is equal to ", x)
8 }, mc.cores = 4)
```

```
[[1]]
[1] "Hello from process #85488. I see x and it is equal to 20"
```

```
[[2]]
[1] "Hello from process #85489. I see x and it is equal to 20"
```

```
[[3]]
[1] "Hello from process #85490. I see x and it is equal to 20"
```

```
[[4]]
[1] "Hello from process #85491. I see x and it is equal to 20"
```

Ex 2: Parallel regressions

Problem: Run multiple regressions on a very wide dataset. We need to fit the following model:

$$y = X_i \beta_i + \varepsilon, \quad \varepsilon \sim N(0, \sigma_i^2), \quad \forall i$$

```
[1] 500 999
```

```
      x001      x002      x003      x004      x005
1  0.61827227  1.72847041 -1.4810695 -0.2471871  1.4776281
2  0.96777456 -0.19358426 -0.8176465  0.6356714  0.7292221
3 -0.04303734 -0.06692844  0.9048826 -1.9277964  2.2947675
4  0.84237608 -1.13685605 -1.8559158  0.4687967  0.9881953
5 -1.91921443  1.83865873  0.5937039 -0.1410556  0.6507415
6  0.59146153  0.81743419  0.3348553 -1.8771819  0.8181764

num [1:500] -0.8188 -0.5438 1.0209 0.0467 -0.4501 ...
```

Ex 2: Parallel regressions (cont. 1)

Serial solution: Use `apply` (basic loop) to solve it

```
1 #  
2 #  
3 #  
4 #  
5 ans <- apply(  
6   X = X,  
7   MARGIN = 2,  
8   FUN = function(x) coef(lm(y ~ x))  
9 )  
10  
11 ans[,1:5]
```

	x001	x002	x003	x004	x005
(Intercept)	-0.03449819	-0.03339681	-0.03728140	-0.03644192	-0.03717344
x	-0.06082548	0.02748265	-0.01327855	-0.08012361	-0.04067826

Ex 2: Parallel regressions (cont. 2)

Parallel solution: Use `parApply`

```
1 library(parallel)
2 cl <- makeCluster(4L)
3 clusterExport(cl, "y")
4 ans <- parApply(
5   cl = cl,
6   X = X,
7   MARGIN = 2,
8   FUN = function(x) coef(lm(y ~ x))
9 )
10
11 ans[,1:5]
```

	x001	x002	x003	x004	x005
(Intercept)	-0.03449819	-0.03339681	-0.03728140	-0.03644192	-0.03717344
x	-0.06082548	0.02748265	-0.01327855	-0.08012361	-0.04067826

Ex 2: Parallel regressions (cont. 3)

Are we going any faster?

```
1 microbenchmark::microbenchmark(  
2   parallel = parApply(  
3     cl = cl,  
4     X = X, MARGIN = 2,  
5     FUN = function(x) coef(lm(y ~ x))  
6   ),  
7   serial = apply(  
8     X = X, MARGIN = 2,  
9     FUN = function(x) coef(lm(y ~ x))  
10  ), unit="relative"  
11 )
```

Unit: relative

	expr	min	lq	mean	median	uq	max	neval
parallel		1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	100
serial		2.643877	2.621755	2.450457	2.59481	2.372613	1.940924	100

Part II: Extended Example

Extended Example: SARS-CoV2 simulation

An altered version of [Conway's game of life](#)

1. People live on a grid, each individual having 8 neighbors.
2. A healthy individual (A) interacting with a sick neighbor (B) has the following probabilities of contracting the disease:
 - a. 100% if neither wears a mask.
 - b. 50% if only A wears a mask.
 - c. 20% if only B wears a mask.
 - d. 5% if both wear masks.
3. Infected individuals may die with probability 10%.

We want to illustrate the importance of wearing face masks.
We are going to simulate a system with 2,500 (50 x 50)

Conway's Game of Masks

Download the program [here](#).

```
1 # source("~/Downloads/sars-cov2.R", echo=FALSE)
2 source("img/sars-cov2.R", echo = FALSE)
3
4 # Looking at some constants
5 probs_sick # Sick individual's probabilities
```

deceased	infected	recovered
0.1	0.4	0.5

```
1 probs_susc # Probabilities of i getting the disease
```

	j doesn't wear	j wears
i doesn't wear	0.9	0.20
i wears	0.5	0.05

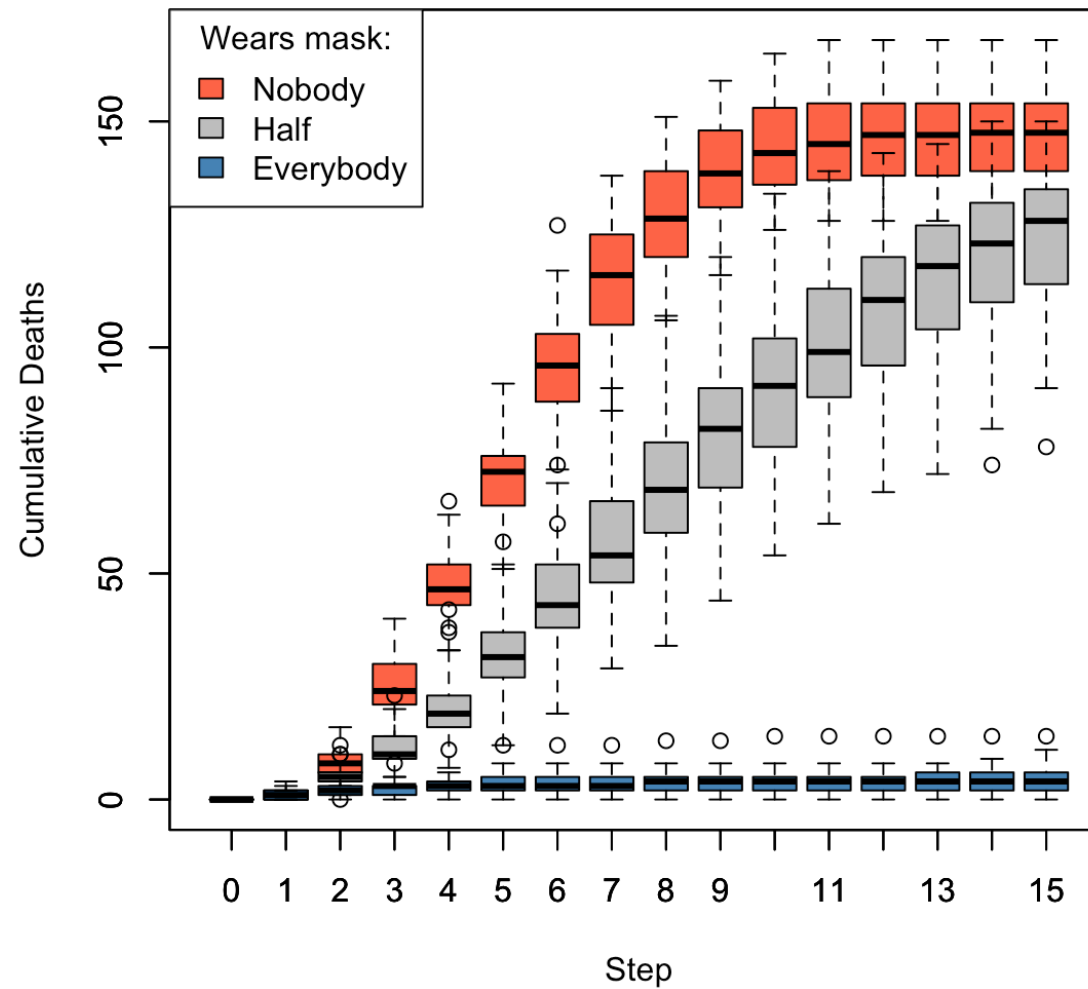
First look: What does the simulation look like?

	susceptible	infected	recovered	deceased
0	1440	160	0	0
1	1265	234	85	16
2	1064	307	190	39
3	876	321	334	69
4	717	287	499	97
15	430	1	990	179
16	429	2	990	179
17	429	0	992	179
18	429	0	992	179
19	429	0	992	179

First look: What does the simulation look like? (contd')

```
1 # Location of who wears the facemask. This step is only for plotting
2 wears <- which(one$wears, arr.ind = TRUE) - 1
3 wears <- wears/(one$nr) * (1 + 1/one$nr)
4
5 # Initializing the animation
6 fig <- magick::image_device(600, 600, res = 96/2, pointsize = 24)
7 for (i in 1:one$current_step) {
8
9   # Plot
10  image(
11    one$temporal[, , i], col=c("gray", "tomato", "steelblue", "black"),
12    main = paste("Time", i - 1L, "of", one$nsteps),
13    xlim = c(1, 4)
14  )
15  points(wears, col="white", pch=20, cex=1.5)
16  legend(
17    "topright",
18    col = c("gray", "tomato", "steelblue", "black", "black"),
19    legend = c(names(codes), "wears a mask"),
```





Cumulative number of deceased as a function of whether none, half, or all individuals wear a face mask.

Speed things up: Timing under the serial implementation

We will use the function `system.time` to measure how much time it takes to complete 20 simulations in serial versus parallel fashion using 4 cores.

Speed things up: Parallel a Forking Cluster

Alternative 1: If you are using Unix-like system (Ubuntu, OSX, etc.), you can take advantage of process forking, and thus, parallel's `mcapply` function:

Speed things up: Parallel with a Socket Cluster

Alternative 2: Regardless of the operating system, we can use a Socket cluster, which is simply a group of fresh R sessions that listen to the parent/main/mother session.

```
1 # Step 2: Prepare the cluster
2 # We could either export all the needed variables
3 parallel::clusterExport(
4   cl,
5   c("calc_stats", "codes", "dat", "get_neighbors", "init", "probs_sick",
6     "probs_susc", "simulate_covid", "update_status", "update_status_all"
7   )
8 )
```

Or simply run the simulation script in the other sessions:

```
1 # Step 2 (alt): Prepare the cluster
2 parallel::clusterEvalQ(cl, source("img/sars-cov2.R"))
3 parallel::clusterSetRNGStream(cl, 123) # Make sure it is reproducible!
```

Bonus: Checking what are the processes running on the system

```
1 (pids <- c(  
2   master    = Sys.getpid(),  
3   offspring = unlist(parallel::clusterEvalQ(cl, Sys.getpid()))  
4 ))  
5 #      master offspring1 offspring2  
6 #      14810      15998      16012
```

Speed things up: Parallel with a Socket Cluster (cont'd)

Using two threads/processes, you can obtain the following speedup

user	system	elapsed
8.987	0.030	9.020

user	system	elapsed
0.001	0.000	4.910

user	system	elapsed
0.001	0.010	4.739

Part III: Cloud Computing

Cloud Computing (a.k.a. on-demand computing)

HPC clusters, super-computers, etc. need not be bought... you can rent:

- [Amazon Web Services \(AWS\)](#)
- [Google Cloud Computing](#)
- [Microsoft Azure](#)

These services provide more than just computing (storage, data analysis, etc.). But for computing and storage, there are other free resources, e.g.:

- The Extreme Science and Engineering Discovery Environment (XSEDE)

There are many ways to run R in the cloud

At USC:

- Center for Advanced Research Computing (CARC). USC users can request hundreds of cores (literally).

Running R in:

- Google Cloud: <https://cloud.google.com/solutions/running-r-at-scale>
- Amazon Web Services: <https://aws.amazon.com/blogs/big-data/running-r-on-aws/>
- Microsoft Azure: <https://docs.microsoft.com/en-us/azure/architecture/data-guide/technology-choices/r-developers-guide>

Submitting jobs

- A key feature of cloud services: interact via command line.
- You will need to be familiar with `Rscript` and `R CMD BATCH`.
- Which is better? It depends on the application.

Submitting jobs (examples)

Imagine we have the following R script (download [here](#)):

```
1 library(data.table)
2 set.seed(1231)
3 dat <- data.table(y = rnorm(1e3), x = sample.int(5, 1e3, TRUE))
4 dat[,mean(y), by = x]
```

R CMD BATCH

This will run a non-interactive R session and put all the output ([stdout](#) and [stderr](#)) to the file [dummy.Rout](#).

```
R CMD BATCH --vanilla dummy.R dummy.Rout &
```

Rscript

This will also execute R in the background, with the difference that the output [dummy.Rout](#) will not capture [stderr](#)

(messages, warnings and errors from R).

```
Rscript --vanilla dummy.R > dummy.Rout &
```

The **&** at the end makes sure the job is submitted and does not wait for it to end.

Rscript

The R script can be executed as a program directly, if you specify where the **Rscript** program lives.

The following example works on Unix. This is an R script named **since_born.R** (download [here](#))

```
1 #!/usr/bin/Rscript
2 args <- tail(commandArgs(), 0)
3 message(Sys.Date() - as.Date(args), " days since you were born.")
```

This R script can be executed in various ways...

Rscript as a program

For this we would need to change it to an executable. In unix you can use the `chmod` command: `chmod +x since_born.R`. This allows us to do the following:

Rscript in a bash script (most common)

In the case of running jobs on a cluster or something similar, we usually need to have a bash script. In our case, we have a file named `since_born_bash.sh` that calls `Rscript` (download [here](#))

```
1 #!/bin/bash
2 Rscript since_born.R 1988-03-02
```

Which we would execute like this:

```
Fatal error: cannot open file 'since_born.R': No such file or directory
```


Summary

- Parallel computing can speed things up.
- It's not always needed... make sure that you are taking advantage of vectorization.
- Most loops can be parallelized (“embarrassingly parallel computing”).
- In R, explicit parallelism can be achieved using the **parallel** package:
 1. Load the package and create a cluster: **library(parallel), parallel::makeCluster()**
 2. Setup the environment: **parallel::clusterExport(), parallel::clusterSetRNGStream()**
 3. Make the call: **parallel::clusterEvalQ(), parallel::parLapply()**

Session info

R version 4.5.1 (2025-06-13)
Platform: aarch64-apple-darwin20
Running under: macOS Sequoia 15.6.1

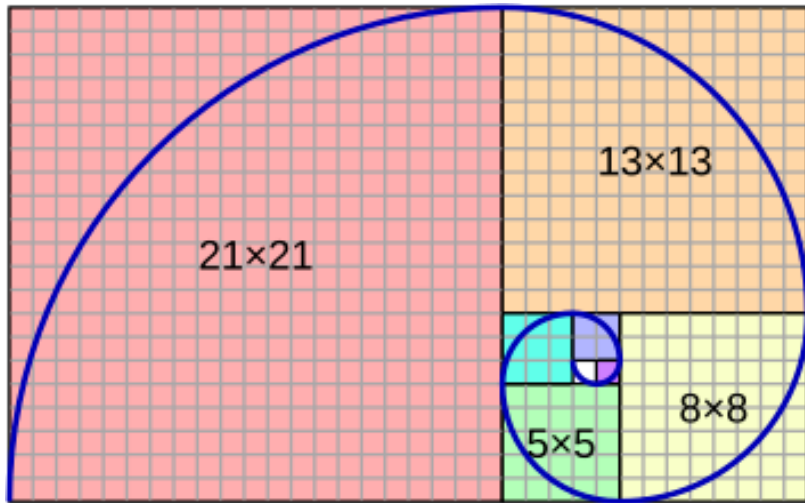
Matrix products: default
BLAS: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK version 3.12.1

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: America/Los_Angeles
tzcode source: internal

(Bonus) **Rcpp**: R & C++

The Fibonacci series



$$\text{Fib}(n) = \begin{cases} n & \text{if } n \leq 1 \\ \text{Fib}(n - 1) + \text{Fib}(n - 2) & \text{otherwise} \end{cases}$$

Rcpp: Hello world! v1

The following C++ file, called `fib.cpp`

Can be compiled within R using

`Rcpp::sourceCpp("fib.cpp")`. This exports the function back into R:

```
[1] 1 1 2 3 5
```

Rcpp: Hello world! v2 (with function overloading)

Rcpp data types are mapped directly to R data types, e.g. vectors of integers in R can be used as `IntegerVector` in Rcpp.

Back in R

```
[1] 1 1 2 3 5
```

RcppArmadillo and OpenMP

- Friendlier than `RcppParallel`... at least for ‘I-use-Rcpp-but-don’t-actually-know-much-about-C++’ users (like myself!).
- Must run only ‘Thread-safe’ calls, so calling R within parallel blocks can cause problems (almost all the time).
- Use `arma` objects, e.g. `arma::mat`, `arma::vec`, etc. Or, if you are used to them `std::vector` objects as these are thread safe.
- Pseudo-Random Number Generation is not very straight forward... But C++11 has a `nice set of functions` that can be used together with OpenMP
- Need to think about how processors work, cache memory, etc. Otherwise you could get into trouble... if your code is slower when run in parallel, then you probably are facing

RcppArmadillo and OpenMP workflow

1. Add the following to your C++ source code to use OpenMP, and tell Rcpp that you need to include that in the compiler:

```
1 #include <omp.h>
2 // [[Rcpp::plugins(openmp)]]
```

2. Tell the compiler that you'll be running a block in parallel with openmp

```
1 #pragma omp [directives] [options]
2 {
3     ...your neat parallel code...
4 }
```

You'll need to specify how OMP should handle the data:

- **shared**: Default, all threads access the same copy.
- **private**: Each thread has its own copy (although not initialized).
- **firstprivate** Each thread has its own copy initialized.

Ex 3: RcppArmadillo + OpenMP

Computing the distance matrix (see `?dist`)

```
1  #include <Rcpp.h>
2  #include <omp.h>
3  #include <RcppArmadillo.h>
4  // [[Rcpp::depends(RcppArmadillo)]]
5  // [[Rcpp::plugins(openmp)]]
6  using namespace Rcpp;
7
8  // [[Rcpp::export]]
9  arma::mat dist_par(const arma::mat & X, int cores = 1) {
10
11     // Some constants and the result
12     int N = (int) X.n_rows; int K = (int) X.n_cols;
13     arma::mat D(N,N,arma::fill::zeros);
14
15     omp_set_num_threads(cores); // Setting the cores
16
17     #pragma omp parallel for shared(D, N, K, X) default(none)
18     for (int i=0; i<N; ++i)
19         for (int j=0; j<i; ++j) {
```

```
1 set.seed(1231)
2 K <- 1000
3 n <- 500
4 x <- matrix(rnorm(n*K), ncol=K)
```

```
1 # Benchmarking!
2 microbenchmark::microbenchmark(
3   dist(x),                # stats::dist
4   dist_par(x, cores = 1), # 1 core
5   dist_par(x, cores = 2), # 2 cores
6   times = 10, unit="relative"
7 )
```

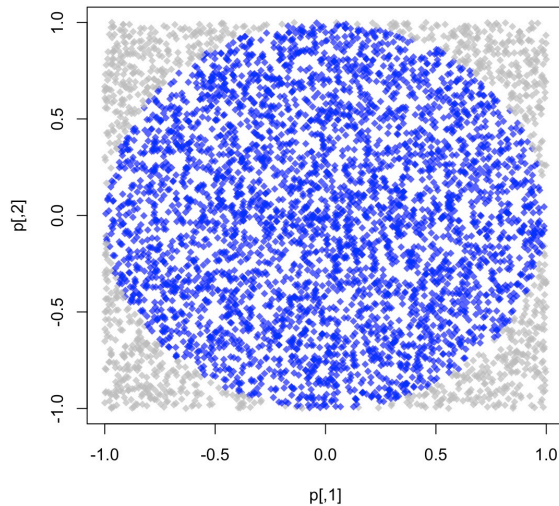
Resources

- [Package parallel](#)
- [Using the iterators package](#)
- [Using the foreach package](#)
- [32 OpenMP traps for C++ developers](#)
- [The OpenMP API specification for parallel programming](#)
- [‘openmp’ tag in Rcpp gallery](#)
- [OpenMP tutorials and articles](#)

For more, checkout the [CRAN Task View on HPC](#)

Simulating π

- We know that $\pi = \frac{A}{r^2}$. We approximate it by randomly adding points x to a square of size 2 centered at the origin.
- So, we approximate π as $\Pr\{\|x\| \leq 1\} \times 2^2$



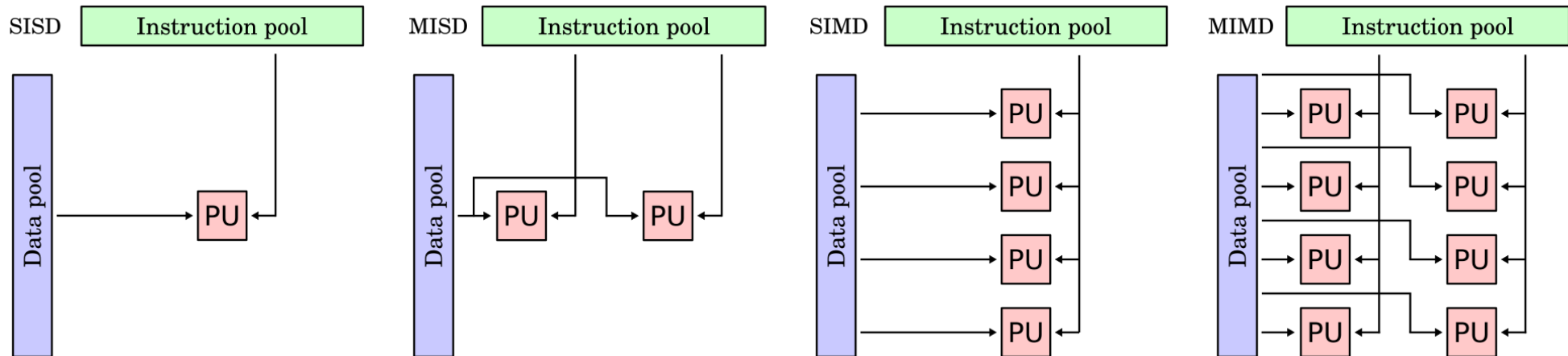
The R code to do this

```
1  pisim <- function(i, nsim) { # Notice we don't use the -i-
2    # Random points
3    ans  <- matrix(runif(nsim*2), ncol=2)
4
5    # Distance to the origin
6    ans  <- sqrt(rowSums(ans^2))
7
8    # Estimated pi
9    (sum(ans <= 1)*4)/nsim
10 }
```


(Bonus) Overview of HPC

Using [Flynn's classical taxonomy](#), we can classify parallel computing according to the following two dimensions:

- a. Type of instruction: Single vs Multiple
- b. Data stream: Single vs Multiple



Michael Flynn's Taxonomy ([wiki](#))

(Bonus) Parallel computing: Software

Implicit parallelization:

- [tensorflow](#): Machine learning framework
- [pqR](#): Branched version of R.
- [Microsoft R](#): Microsoft's R private version (based on Revolution Analytics' R version).
- [data.table](#) (R package): Data wrangling using multiple cores.
- [caret](#) (R package): A meta package, has various implementations using parallel computing.

Explicit parallelization ([DIY](#)):

- [CUDA](#) (C/C++ library): Programming with GP-GPUs.
- [Open MP](#) (C/C++ library): Multi-core programming (CPUs).
- [Open MPI](#) (C/C++ library): Large scale programming with multi-node systems.
- [Threading Building Blocks](#) (C/C++ library): Intel's parallel computing library.
- [Kokkos](#) (C++ library): A hardware-agnostic programming framework for HPC applications.
- [parallel](#) (R package): R's built-in parallel computing package
- [future](#) (R package): Framework for parallelizing R.
- [RcppParallel](#) (R C++ API wrapper): Header and templates for building [Rcpp](#)+multi-threaded programs.

- [julia](#) (programming language): High-performing, has a framework for parallel computing as well.